This rubric is designed for a **2-week AI-Assisted Laravel Development Project** where students are expected to leverage AI, agentic workflows, prompt engineering, and Laravel best practices.

Final Project

AI-Assisted Student Information Management System (Web + REST API)

Project Overview

Develop a Student Information Management System using Laravel that demonstrates mastery of Modules 1-10 and incorporates modern AI-assisted development practices.

Students are allowed and encouraged to use:

- ChatGPT
- Claude
- Gemini
- GitHub Copilot
- Cursor
- Windsurf
- Cline
- Roo Code
- OpenHands
- Other AI Development Platforms

However, students will be graded not only on the final system but also on:

- AI workflow design
- Prompt engineering quality
- Agent design
- Validation of AI output
- Documentation
- Engineering decisions

Required Features

Core Web Application

Authentication

- User Registration
- User Login
- User Logout

Student Management

- Create Student
- View Students
- Edit Student
- Delete Student

Subject Management

- Create Subject
- View Subject
- Edit Subject
- Delete Subject

Enrollment Module

- Enroll Student
- View Enrollments
- Update Enrollment
- Remove Enrollment

Validation

- Server-side Validation
- Error Messages

Authorization

- Admin-only features
- Protected routes

REST API

Must provide:

```
GET /api/students
```

```
GET /api/students/{id}

POST /api/students

PUT /api/students/{id}

DELETE /api/students/{id}
```

Additional endpoints are encouraged.

Required Documentation

Students must submit:

1. Running Application

Laravel project

---

2. AI Engineering Portfolio

File:

```
ai-portfolio.pdf
```

Contents:

- AI tools used
- Model(s) used
- Prompt history
- Prompt iterations
- Screenshots

---

3. Agent Design Document

File:

```
agent-design.pdf
```

Contents:

- Agent architecture
- Agent roles
- Context management

- Token management

---

4. Reflection Report

File:

```
reflection.pdf
```

Discuss:

- What worked
- What failed
- AI mistakes
- Human corrections
- Lessons learned

Detailed Grading Rubric

A. Laravel Technical Implementation (40%)

A1. Authentication (5%)

Level	Criteria
Excellent (5)	Complete login, registration, logout functionality working correctly
Good (4)	Minor issues present
Fair (3)	Core functionality works with several issues
Poor (1-2)	Authentication incomplete

A2. Authorization (5%)

Level	Criteria
Excellent (5)	Proper role protection and route restrictions
Good (4)	Small security gaps
Fair (3)	Limited authorization implementation
Poor (1-2)	Authorization missing

A3. CRUD Implementation (10%)

Level	Criteria
Excellent (10)	All CRUD operations fully implemented and tested
Good (8)	Minor issues only
Fair (6)	CRUD partially complete

Poor (1-5)	Significant missing functionality

A4. Database Design & Eloquent (10%)

Level	Criteria
Excellent (10)	Proper migrations, relationships, Eloquent usage
Good (8)	Minor schema issues
Fair (6)	Database functional but poorly designed
Poor (1-5)	Weak implementation

A5. REST API Implementation (10%)

Level	Criteria
Excellent (10)	Complete REST endpoints with correct responses
Good (8)	Minor inconsistencies
Fair (6)	Basic endpoints only
Poor (1-5)	Missing API features

B. Software Engineering Quality (15%)

B1. MVC Compliance (5%)

Level	Criteria
Excellent (5)	Proper separation of concerns
Good (4)	Minor architectural issues
Fair (3)	Some logic misplaced
Poor (1-2)	MVC poorly applied

B2. Code Quality (5%)

Level	Criteria
Excellent (5)	Clean, organized, readable code
Good (4)	Minor inconsistencies
Fair (3)	Moderate code smells
Poor (1-2)	Difficult to maintain

B3. Project Structure (5%)

Level	Criteria
Excellent (5)	Consistent naming and organization
Good (4)	Minor structure issues
Fair (3)	Inconsistent organization

Poor (1-2)	Poor structure

C. AI Engineering Competency (25%)

C1. Prompt Engineering Quality (10%)

Students must submit prompt history.

Level	Criteria
Excellent (10)	Prompts are precise, iterative, contextual, well-engineered
Good (8)	Generally effective prompts
Fair (6)	Moderate prompting skill
Poor (1-5)	Basic copy-paste prompting

Evidence Required

Examples:

```
Initial Prompt

Refined Prompt

Validation Prompt

Debug Prompt
```

Instructor evaluates prompt maturity.

C2. Agent Design Quality (10%)

Students must show purposeful agent design.

Example

```
Planner Agent

Laravel Developer Agent

Database Architect Agent

Testing Agent

Documentation Agent
```

Level	Criteria
Excellent (10)	Clear specialized multi-agent workflow
Good (8)	Multiple agents with defined responsibilities
Fair (6)	Limited specialization
Poor (1-5)	Single-agent workflow only

C3. Token & Context Management (5%)

Student must demonstrate understanding of:

```
Context windows

Prompt chunking

Task decomposition

Context reuse
```

Level	Criteria
Excellent (5)	Efficient AI workflow with clear context management
Good (4)	Minor inefficiencies
Fair (3)	Basic awareness
Poor (1-2)	No consideration of token usage

D. Testing, Verification, and Validation (10%)

D1. Validation & Error Handling (5%)

Level	Criteria
Excellent (5)	Robust validation and error handling
Good (4)	Minor issues
Fair (3)	Limited validation
Poor (1-2)	Validation mostly absent

D2. AI Output Verification (5%)

Students must document:

```
AI suggested X

Problem found

```

```
Correction applied

Result validated
```

Level	Criteria
Excellent (5)	Strong evidence of critical evaluation
Good (4)	Some validation shown
Fair (3)	Minimal validation
Poor (1-2)	Blind acceptance of AI output

E. Documentation and Reflection (10%)

E1. Technical Documentation (5%)

Must include:

- Setup Instructions
- Database Setup
- API Usage
- Screenshots

Level	Criteria
Excellent (5)	Professional documentation
Good (4)	Mostly complete
Fair (3)	Missing sections
Poor (1-2)	Poor documentation

E2. Reflection Report (5%)

Must discuss:

```
AI Tools Used

Prompt Strategy

Agent Strategy

Challenges

Corrections

```

```
11  Lessons Learned
```

Level	Criteria
Excellent (5)	Deep reflection and insight
Good (4)	Good explanation
Fair (3)	Surface-level discussion
Poor (1-2)	Minimal reflection

Bonus Points (+10 Maximum)

Multi-Agent Architecture (+3)

Demonstrates:

```
1  Planner Agent
2
3  Builder Agent
4
5  Reviewer Agent
6
7  Documentation Agent
```

AI Comparison Study (+2)

Compare outputs from:

```
1  GPT-5
2
3  Claude
4
5  Gemini
6
7  Copilot
```

and justify choices.

API Documentation (+1)

Swagger/OpenAPI/Postman Collection.

Automated Testing (+2)

Laravel Feature or Unit Tests.

Deployment (+2)

Successfully deployed online:

```
Railway

Render

Laravel Cloud

VPS
```

Final Score Summary

Area	Weight
Laravel Technical Implementation	40%
Software Engineering Quality	15%
AI Engineering Competency	25%
Testing & Validation	10%
Documentation & Reflection	10%
TOTAL	100%
Bonus	+10%

What Earns an "A" (90-100)

An A-level submission demonstrates:

- ✅ Complete Student Information Management System
- ✅ Proper MVC and Laravel practices
- ✅ Working REST APIs
- ✅ Authentication and Authorization
- ✅ Well-designed multi-agent workflow
- ✅ Documented prompt engineering process
- ✅ Evidence of AI output verification
- ✅ Professional documentation
- ✅ Reflection showing engineering judgment rather than simply generating code with AI

This rubric rewards not only **what students build**, but **how they collaborate with AI as software engineers**, which aligns extremely well with your inclusion of Agentic Development as a capstone topic.